INVYSTA

AAK Device Authentication Embedded Service Deployment Guide

Technical reference for client engineering and operations teams

v1.0 · May 2026

- No source code exposure — fully protected IP
- Self-hosted or Invysta-hosted deployment
- Full data residency within your environment
- Purpose-built client apps for iOS, Android & Windows
- EU regulatory and governance aligned

TABLE OF CONTENTS

1. Overview

This guide describes the **Embedded Service** integration model for Invysta AAK (Anonymous Access Key) device authentication. It is an alternative to the managed OAuth/OIDC service, suited to organizations that prefer a simpler, more direct integration without the overhead of an OAuth/OIDC flow, as well as those that require full operational control, data residency within their own environment, or integration into regulated platforms where third-party cloud dependencies are restricted or prohibited.

Invysta delivers a **service** and **platform-native client applications**. Your platform invokes two straightforward APIs — Device Registration and Device Authentication — and Invysta handles all cryptographic complexity within the service.

> **How this differs from the managed OAuth/OIDC service:** In the OAuth/OIDC model, Invysta operates the authentication infrastructure as a cloud service and your application delegates to it via standard OIDC redirects. With this model, the Invysta authentication service runs in your infrastructure or as a service managed by Invysta. You control the execution environment, the data, and the network boundary — Invysta cannot access any of these without your involvement.

2. Deployment Options

The embedded service deployment model supports two hosting arrangements. Both use identical APIs and client applications — the only difference is who operates the infrastructure.

Self-Hosted

You deploy and operate the Invysta service within your own infrastructure. The service runs on hardware and an OS environment you control, in a data center or cloud tenancy of your choosing. All authentication data remains within your network boundary and never leaves your environment.

> **Best for:** Regulated industries (finance, energy, healthcare), platforms with EU or national data residency obligations, organizations with existing internal infrastructure and operations teams, or environments where third-party cloud dependencies are restricted by policy or contract.

Invysta-Hosted

Invysta operates the Invysta service on your behalf within a dedicated, isolated hosting environment. You access the same two APIs over a private, secured connection. Invysta manages deployment, patching, scaling, and high availability. Your client applications and your platform code are unchanged regardless of which

hosting arrangement you choose.

> **Best for:** Organizations that want the control and security of the service model without the operational overhead of running the service themselves. Invysta-hosted instances can be provisioned in specific geographic regions to meet data residency requirements.

Comparison

	Self-Hosted	Invysta-Hosted
Infrastructure	You provide and manage	Invysta provides and manages
Data residency	Fully within your environment	Dedicated isolated instance, region of your choice
Operational responsibility	Your team	Invysta SRE team
Patching & updates	Invysta delivers service updates; you deploy	Invysta deploys automatically
High availability	You architect HA	Invysta provides HA by default
APIs & client apps	Identical	Identical
Network path	Internal only	Traffic routed over a private network connection — VPN tunnel, VPC peering, or dedicated interconnect. Never traverses the public internet.

3. Architecture

The integration uses a **split architecture**: a backend service and platform-native client applications that work together to perform device-bound authentication. Neither component alone can complete an authentication — the cryptographic exchange requires both sides.

Backend — The Invysta Binary Service

A self-contained service deployed within your infrastructure (or Invysta's hosted environment). It exposes two internal APIs — Device Registration and Device Authentication — that your platform calls as part of its authentication workflow. The service is **internal-only** and must not be exposed to external networks or end users.

Property	Detail
Delivery format	Compiled service (platform-specific build delivered by Invysta)
Deployment	VM, container (Docker/Kubernetes), or bare-metal — your choice
Database	Encrypted PostgreSQL (default). Alternative databases available on request.
Network exposure	Internal only — never exposed to external networks or end users
Source code	Not provided. Invysta IP remains fully protected.
Updates	Invysta delivers new service versions. You deploy at your own schedule.

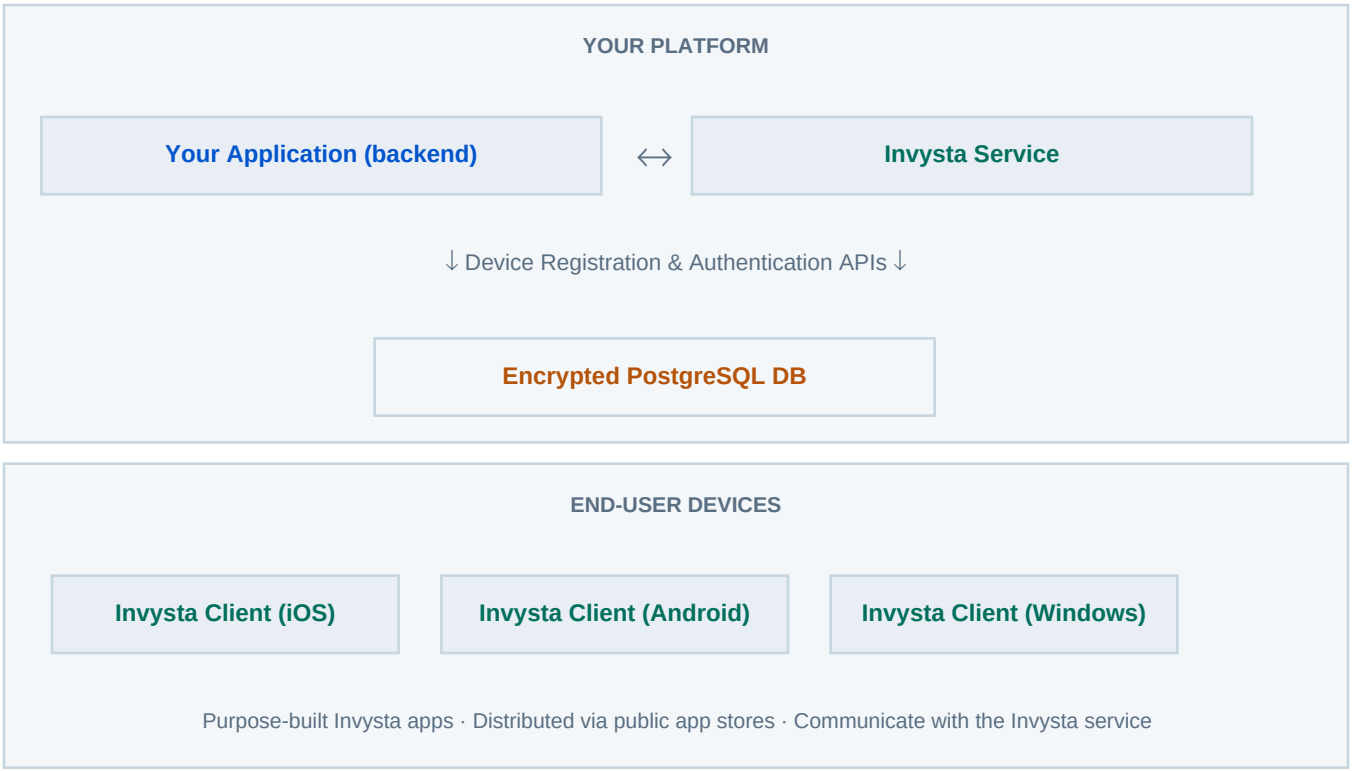
Client — Platform-Native Applications

Invysta provides purpose-built client applications for iOS, Android, and Windows. These apps run on end-user devices, collect device-specific identifiers and signals, and respond to server-initiated authentication challenges. They are distributed via official public app stores, ensuring trusted delivery and automatic updates without any distribution overhead for your team.

Platform	Distribution Channel
iOS	Apple App Store
Android	Google Play Store
Windows	Microsoft Store

Why app store distribution matters: Official app stores provide cryptographic signing, tamper detection, and automatic update delivery. Your platform has no software distribution overhead — users receive updates transparently.

Architecture Overview



4. What Invysta Delivers

Deliverable	Description
Invysta service (compiled)	Platform-specific build of the Invysta AAK authentication service. Includes installer and configuration reference.
Client applications	Purpose-built Invysta apps for iOS, Android, and Windows. Distributed via official public app stores.
API definitions	Full specification for the Device Registration and Device Authentication proxy endpoints, including response codes and the JWKS endpoint for encryption key retrieval.
Integration guidance	This document, plus onboarding support from Invysta engineers.
Service updates	Invysta delivers updated service versions as security patches or capability improvements are released. Self-hosted customers deploy at their own schedule.

[1] Client applications are delivered as purpose-built Invysta apps. Optional white-label branding — including your logo, color scheme, and UX styling — is available as an additional service. Contact your Invysta account manager for details.

5. What Your Team Provides

For **self-hosted** deployments, your team provides the infrastructure environment and integrates the two APIs into your authentication workflow. For **Invysta-hosted** deployments, only the API integration is required.

Self-Hosted Infrastructure Requirements

Requirement	Detail
Hardware specification	CPU, RAM, and storage sizing provided by Invysta based on expected user volume. Share projected concurrent users at onboarding.
Operating system	Linux (Ubuntu 22.04 LTS or RHEL 8+ recommended). Windows Server available on request.
Deployment model	VM, Docker container, or Kubernetes pod. Invysta provides configuration for each.
Database	PostgreSQL 14+ (managed or self-hosted). Invysta configures the schema and encryption at rest.

Requirement	Detail
Network	Internal network access from your application servers to the Invysta service. The JWKS endpoint must also be reachable by the Invysta client apps on end-user devices for public key retrieval. No other inbound external access is required.
TLS certificate	Required for the Invysta service endpoint. Can be internal CA or public certificate.

API Integration

Regardless of hosting arrangement, your platform backend must call two APIs: Device Registration (once per device, per user) and Device Authentication (on every login). Both are simple JSON over HTTPS. See Sections 6 and 7.

6. Device Registration

Device registration occurs once per device, per user — typically when a user first installs the Invysta client app and completes the enrollment flow. Your host server acts as a proxy: it receives the registration request from the client app and forwards it, unchanged, to the Invysta service. All validation, fingerprint storage, and device association logic is handled entirely within the Invysta service.

The registration payload is encrypted by the client app using the Invysta service public key before it is sent to the host server. The host server therefore proxies ciphertext — it cannot read, inspect, or log the payload contents. Only the Invysta service, holding the corresponding private key, can decrypt and process it.

Proxy Flow

1

Client app encrypts and sends registration payload

The Invysta client app collects device identifiers, assembles the registration payload, and encrypts it using the Invysta service public key (hybrid encryption: RSA-OAEP envelope with AES-GCM payload). The resulting ciphertext is sent to the host server. The plaintext payload never exists outside the client app.

2

Host server blind-proxies ciphertext to the Invysta service

Your host server forwards the encrypted payload, unchanged, to the Invysta service endpoint. The host cannot decrypt or inspect the contents — it is forwarding opaque ciphertext.

```
POST https:///api/v1/device/register
Authorization: Bearer // see Section 8 for auth options
Content-Type: application/json
```

3

Invysta service validates and responds

The Invysta service performs all proprietary validation logic and returns a result to your host server. Your host server acts on the response code and relays the outcome to the client app.

Response Codes

HTTP Code	Meaning	Your Platform Action
`201 Created`	Device registered successfully	Confirm enrollment to the user. Registration complete.
`401 Unauthorized`	Invalid one-time code or credentials	Prompt the user to check their enrollment code and retry.

HTTP Code	Meaning	Your Platform Action
`403 Forbidden`	Device limit reached for this user account	Inform the user they have reached their device limit. Offer a device management flow to deregister an existing device.

Device limits: Each user account has a maximum number of registered devices set by your Invysta license. The limit is configurable at onboarding. Your platform can surface a device management screen allowing users to deregister old devices before adding new ones.

7. Device Authentication

Device authentication is called on every login attempt. Your host server issues an authentication challenge to the Invysta client app, then proxies the app's response to the Invysta service. The service performs all proprietary fingerprint matching and validation logic, and returns a pass or fail decision. Your host server grants or denies the login based solely on that response code.

As with registration, the authentication response payload is encrypted by the client app using the Invysta service public key before it reaches the host server. The host proxies ciphertext end-to-end — the device fingerprint and AAK are never exposed in plaintext to the host server or any intermediary.

Proxy Flow

1 **Host server issues authentication challenge**

When a user attempts to log in, your host server issues an authentication challenge to the Invysta client app on the user's device. The challenge format is provided by Invysta during integration.

2 **Client app encrypts and sends authentication response**

The Invysta client app collects the current device fingerprint, assembles the authentication response, and encrypts it using the Invysta service public key. The resulting ciphertext is sent to the host server. The fingerprint data never exists in plaintext outside the client app.

3 **Host server blind-proxies ciphertext to the Invysta service**

Your host server forwards the encrypted authentication response, unchanged, to the Invysta service endpoint. The host cannot decrypt or inspect the contents — it is forwarding opaque ciphertext.

```
POST https:///api/v1/device/authenticate
Authorization: Bearer // see Section 8 for auth options
Content-Type: application/json
```

4 **Invysta service validates and responds**

The Invysta service performs all proprietary matching and validation logic and returns a result to your host server. Your host server acts on the response code — granting or denying the login. The Invysta service does not issue session tokens; session management remains your platform's responsibility.

Response Codes

HTTP Code	Meaning	Your Platform Action
`200 OK`	Device verified — authentication approved	Grant the user access. Create a session, issue your own token, or continue your existing auth flow.
`401 Unauthorized`	Device not recognized — fingerprint did not match any registered device	Deny the login. Return a generic error to the user. Log the full response for diagnostics.
`403 Forbidden`	Account suspended or not authorized for device authentication	Deny the login. Return a generic error to the user. Do not surface the specific reason to the end user.

User-facing error messages: Do not surface raw HTTP codes or Invysta error descriptions to end users. Use a generic message such as "Authentication failed. Please try again or contact support." Log the full Invysta response internally for diagnostics and audit.

8. Securing API Calls to the Binary Service

The Invysta service is internal-only, but calls to it from your platform must still be authenticated to prevent unauthorized use by other services or processes within your network. Three approaches are available, in ascending order of security strength.

Tier 1 — Shared Secret (API Key)

A pre-shared key is generated at deployment and stored as an environment variable in both your application and the Invysta service. Your application includes it in every request as a header:

```
X-Invysta-API-Key:
```

Property	Detail
Complexity	Low — single environment variable on each side
Strength	Adequate for internal-only services in controlled networks
Risk	If the key leaks (logs, env dumps), all access is compromised until rotated
Rotation	Manual. Establish a rotation schedule (e.g. every 90 days) and document the procedure
Recommended for	Development environments, low-risk internal platforms, or where simplicity is the priority

Tier 2 — JWT Bearer Token (Recommended)

> **Recommended for production platforms, especially regulated or high-value environments.**

Your application generates a short-lived signed JWT and includes it in the Authorization header of every request to the service:

```
Authorization: Bearer eyJhbGciOiJSUzI1NiJ9...
```

The JWT payload contains:

```
{
"iss": "your-platform-app", // issuer: identifies your application
"aud": "invysta-auth-service", // audience: the Invysta service
"scope": "device:register device:authenticate",
```

```
"iat": 1716000000, // issued-at timestamp
"exp": 1716003600 // expiry: short window, e.g. 5-60 minutes
}
```

The service validates the token signature, expiry, and audience claim before processing any request. An expired or invalid token is rejected with HTTP 401.

Signing options:

Signing Method	How It Works	Strength	Recommended
HMAC-SHA256 (HS256)	Both sides share a signing secret	Good	Dev / simple deployments
RSA-SHA256 (RS256)	Your app signs with private key; the service verifies with public key. Private key never leaves your environment.	Strong	Production — recommended
ECDSA (ES256)	Same as RS256 but smaller keys and faster verification	Strong	Production — preferred for high-throughput

Property	Detail
Complexity	Low-medium — a few lines of code using any JWT library
Strength	Strong. Tokens expire automatically; no long-lived credential to steal
Scope control	Issue separate tokens for registration vs. authentication calls if desired
Key rotation	Rotate signing keys without service restart; the service picks up new public key on next validation
Recommended for	All production deployments

Tier 3 — Mutual TLS (mTLS)

Both your application and the Invysta service present TLS certificates to each other. The service only accepts connections from clients presenting a certificate signed by a trusted CA. This verifies the identity of the caller at the transport layer, independent of any application-level credential.

```
# Your application presents a client certificate on every connection
curl --cert client.crt --key client.key \
--cacert invysta-ca.crt \
https:///api/v1/device/authenticate
```

Property	Detail
Complexity	High — requires PKI infrastructure, certificate issuance, and lifecycle management
Strength	Maximum. Identity verified at transport layer; no application credential to intercept
Best for	Environments with existing PKI (e.g. enterprise, financial, government), or where compliance mandates mTLS
Can be combined with	JWT bearer tokens for defense-in-depth (mTLS at transport + JWT at application layer)

Recommendation summary: Start with JWT bearer tokens (Tier 2, RS256 signing) for all production deployments. Add mTLS (Tier 3) if your environment already has PKI infrastructure or if compliance requirements mandate it. Use shared secret (Tier 1) only for development and testing.

9. Security Model

Property	Detail
End-to-end payload confidentiality	Device fingerprint and AAK data is encrypted by the client app using the Invysta service public key (RSA-OAEP + AES-GCM) before leaving the device. The client app retrieves the encryption public key from the Invysta JWKS endpoint at startup, identified by use=enc. The host server proxies ciphertext and cannot read, inspect, or log the payload. Only the Invysta service, holding the private key, can decrypt it. Key rotation is handled automatically via the kid field.
Cryptographic split	Sensitive operations are divided between the device-side client app and the Invysta service. Neither component alone can authenticate a user.
No source code exposure	The service is delivered compiled. No Invysta algorithms, source code, or private keys are visible to your team or environment.
Device-bound authentication	Authentication is tied to a physical device, not a reusable credential. A stolen password alone cannot grant access.
Encrypted data store	All device and authentication metadata is stored in an encrypted PostgreSQL database within your environment.
Internal-only service	The service API endpoints must not be exposed to external networks. Only your application backend should be able to reach it.
No data to Invysta	In self-hosted deployments, no authentication data is transmitted to or stored in Invysta-managed infrastructure.
App store distribution	Purpose-built Invysta client apps are distributed via Apple, Google, and Microsoft stores, providing cryptographic signing, integrity verification, and automatic updates.

10. Data Residency & Operational Control

Item	Self-Hosted	Invysta-Hosted
Authentication metadata	Stored exclusively in your environment	Stored in dedicated isolated instance, region of your choice
Data transmitted to Invysta	None	None — data stays within the dedicated instance
EU / national residency	Fully compliant — you choose the location	Compliant — Invysta provisions in your required region

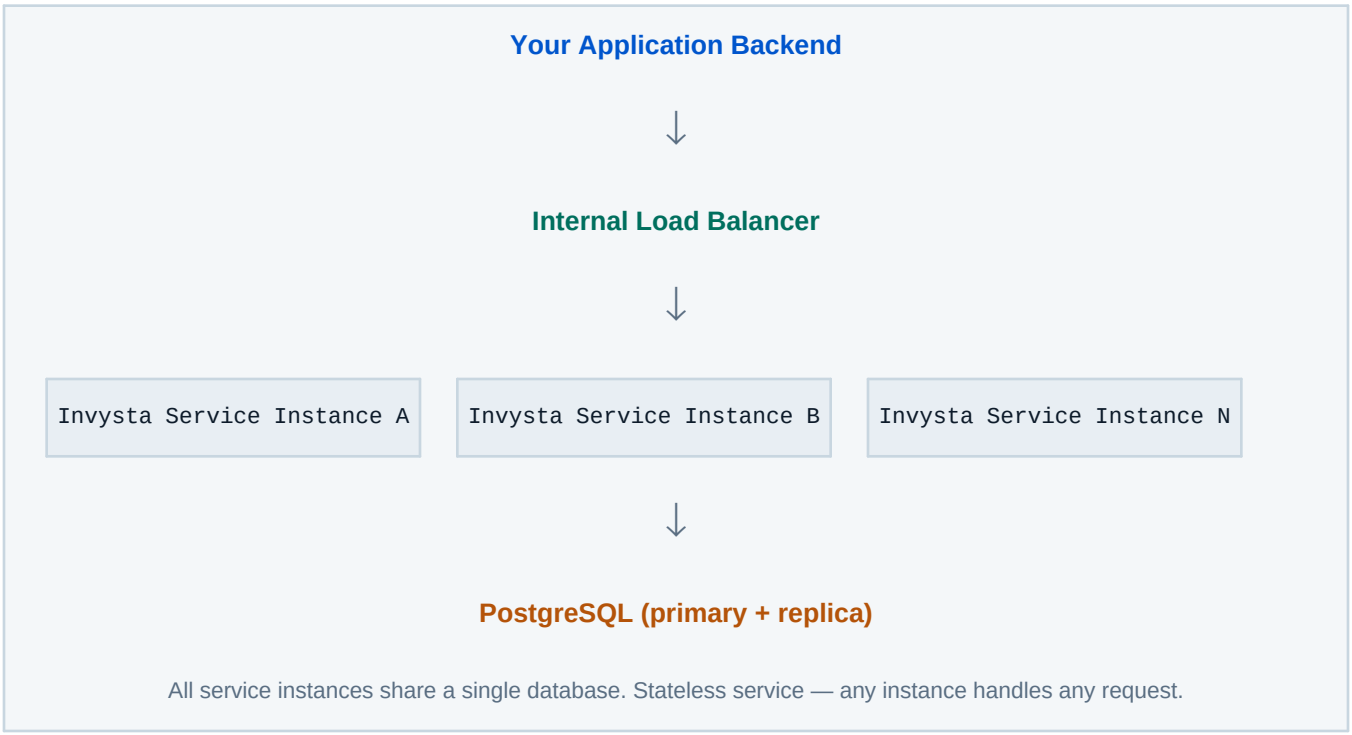
Item	Self-Hosted	Invysta-Hosted
Operational ownership	Your team	Invysta SRE team, under agreed SLA
Audit logs	Stored in your environment	Accessible to you via API or dashboard
Regulatory frameworks	GDPR, NIS2, sector-specific (finance, energy, healthcare)	Same — Invysta provides compliance documentation on request

11. Fault Tolerance & High Availability

For **Invysta-hosted** deployments, high availability is provided by default — multiple service instances behind a load balancer with a shared session store. Planned maintenance causes zero downtime.

For **self-hosted** deployments, your team architects HA using standard patterns. The service is stateless between requests (state is persisted to PostgreSQL), so horizontal scaling is straightforward.

Recommended Self-Hosted HA Pattern



Impact on your platform if the Invysta service is unreachable: New login attempts requiring device authentication will fail. Existing sessions your platform has already established are unaffected — the Invysta service is only involved at the point of authentication, not for session validation. Implement a circuit-breaker pattern in your integration to handle service unavailability gracefully.

12. Integration Checklist

✓

Choose your deployment model

Self-hosted or Invysta-hosted. Communicate your choice and any data residency requirements to Invysta at onboarding.

✓

Provision infrastructure (self-hosted only)

Share hardware specs, OS, and preferred deployment model (VM / Docker / Kubernetes) with Invysta. Provision PostgreSQL database.

✓

Receive and deploy the service

Invysta delivers the service package and configuration reference. Deploy to your environment and verify the service starts correctly.

✓

Choose your API authentication tier

Shared secret (Tier 1) for development. JWT bearer RS256 (Tier 2) for production. mTLS (Tier 3) for high-security or compliance-mandated environments.

See Section 8 for implementation details.

✓

Integrate Device Registration API

Your platform backend calls POST /api/v1/device/register when a user completes the client app enrollment flow. Handle 201, 401, and 403 responses.

See Section 6.

✓

Integrate Device Authentication API

Your platform backend calls POST /api/v1/device/authenticate on every login. Grant access on 200, deny on 401/403.

See Section 7.

✓

Deploy Invysta client apps

Invysta submits purpose-built client apps to the Apple App Store, Google Play, and Microsoft Store. Coordinate app store accounts and review timelines with Invysta at onboarding.

✓

Test end-to-end

Enroll a test device, authenticate, verify success and failure paths. Test HA failover if self-hosting.

✓

Go live

Roll out to users. Invysta provides onboarding support throughout.

No Invysta source code in your environment. No authentication data leaving your network.